

Tyler Vander Mooren

ITC 413: NoSQL Databases

Professor Bryan Knueppel

March 7, 2026

Assignment 8

Part 1: Daily Active Users

Screenshot 1: Query to get total weekly active users.

```
> SMEMBERS analytics:wau:week1
1) "player:alice"
2) "player:bob"
3) "player:carol"
4) "player:david"
```

1 Command(s) - 1 success, 0 error(s)

13:38:37 5 Mar 2026

```
> SCARD analytics:wau:week1
(integer) 4
```

I track daily active users in Sets above. Weekly Active Users (WAU) are computed comparing the five daily sets with SUNIONSTORE and counting unique players using SCARD.

Screenshot 2: Analysis of which players are most consistent

5 Command(s) - 5 success, 0 error(s)

13:41:12 5 Mar 2026

39.31 msec

```
> SISMEMBER analytics:active:2024-03-11 player:alice
(integer) 1
> SISMEMBER analytics:active:2024-03-12 player:alice
(integer) 1
> SISMEMBER analytics:active:2024-03-13 player:alice
(integer) 1
> SISMEMBER analytics:active:2024-03-14 player:alice
(integer) 0
> SISMEMBER analytics:active:2024-03-15 player:alice
(integer) 1
```

Consistency is measured across the 5 days. Alice is active on 4/5 days, Carol and David on 3/5 and Bob on 2/5, so Alice is the most consistent player.

Part 2: Player Retention Analysis

Screenshot 3 & 4: System to record new player registrations

```
> SADD analytics:new_players:2024-03-11 player:alice player:bob
(integer) 2
> SADD analytics:new_players:2024-03-12 player:carol
(integer) 1
> SADD analytics:new_players:2024-03-13 player:david
(integer) 1
```

```
> SMEMBERS analytics:new_players:2024-03-11
1) "player:alice"
2) "player:bob"
```

New registrations are stored in the Sets Above then updated with SADD and checked with SMEMBERS. Retention is calculated by comparing the original group with players that are active on later days.

Screenshot 5: List of players with longest login streaks

```
1 Command(s) - 1 success, 0 error(s) 13:56:15 5 Mar 2024

> ZREVRANGE analytics:streaks 0 -1 WITHSCORES
1) "player:david"
2) "3"
3) "player:alice"
4) "3"
5) "player:carol"
6) "2"
7) "player:bob"
8) "1"

1 Command(s) - 1 success, 0 error(s) 13:56:12 5 Mar 2024

> ZADD analytics:streaks 3 player:alice 1 player:bob 2 player:carol 3 player:david
(integer) 4
```

Login streaks are recorded in hashes. A Sorted Set holds each player's max streak and ranks them. David and Alice have streaks of 3, Carol has 2 and Bob has 1, so David and Alice are the most loyal players.

Part 3: Revenue Tracking

Screenshot 6: Daily revenue tracking system

```
> GET revenue:daily:2024-03-11
"10"
> GET revenue:daily:2024-03-12
"12"
> GET revenue:daily:2024-03-13
"10"
> GET revenue:daily:2024-03-14
"6"
> GET revenue:daily:2024-03-15
"8"

30 Command(s) - 30 success, 0 error(s)

> INCRBY revenue:daily:2024-03-11 5
(integer) 5
> ZINCRBY revenue:players 5 player:alice
"5"
> INCRBY revenue:category:gold_pack 5
(integer) 5
> INCRBY revenue:daily:2024-03-11 3
(integer) 8
> ZINCRBY revenue:players 3 player:bob
"3"
> INCRBY revenue:category:skins 3
```

Daily revenue is stored as string counters with keys. Each in-app purchase increments the appropriate day using INCRBY. Calling GET revenue:daily:<date> returns the total revenue for that day.

Screenshot 7: Player spending analysis

```
> ZREVRANGE revenue:players 0 -1 WITHSCORES
1) "player:alice"
2) "18"
3) "player:carol"
4) "12"
5) "player:bob"
6) "9"
7) "player:david"
8) "7"
```

Player spending is tracked in the Sorted Set revenue:players, where the score is total lifetime spend. Above command returns a top-spenders leaderboard. In my data, Alice (18) and Carol (12) are the biggest contributors, followed by Bob (9) and David (7), showing that a few players generate most of the revenue.

Part 4: Feature Usage Analytics

Screenshot 8: Feature usage counters and rankings

```
> HGETALL analytics:features:global
1) "leaderboard_views"
2) "15"
3) "profile_updates"
4) "5"
5) "settings_changes"
6) "3"
7) "friends_actions"
8) "6"
9) "messages_sent"
10) "13"

1 Command(s) - 1 success, 0 error(s)

> ZREVRANGE analytics:game_modes 0 -1 WITHSCORES
1) "ranked"
2) "41"
3) "casual"
4) "27"
5) "practice"
6) "10"
```

Global feature usage is stored in the hash analytics:features:global, where the fields are incremented with HINCRBY. HGETALL shows which features are used most. Game modes are ranked in the Sorted Set analytics:game_modes; ZREVRANGE shows that ranked mode is the most popular, followed by casual and practice.

Screenshot 9: Per-player feature engagement profiles

```
> HGETALL analytics:features:player:alice
1) "leaderboard_views"
2) "5"
3) "profile_updates"
4) "2"
5) "messages_sent"
6) "4"
> HGETALL analytics:features:player:bob
1) "leaderboard_views"
2) "2"
3) "settings_changes"
4) "1"
5) "messages_sent"
6) "1"
```

Per-player engagement is tracked in hashes. Each hash records counts for the same core features. For example, Alice generates many leaderboard views and profile updates, while Carol has a high messages_sent value, indicating heavier use of social features.

Part 5: Performance Monitoring

Screenshot 10: System metrics dashboard

```
> HGETALL metrics:system:current
1) "online_players"
2) "120"
3) "active_games"
4) "34"
5) "avg_response_ms"
6) "85"
> HGETALL metrics:system:2024-03-05T14
1) "online_players"
2) "120"
3) "active_games"
4) "34"
5) "avg_response_ms"
6) "85"
> GET alerts:slow_response
"avg_response_ms > 200 at 2024-03-05T15"
```

System health is monitored with Redis hashes. metrics:system:current holds live values and is queried with HGETALL for a quick dashboard view. Historical snapshots use time-stamped hashes, so performance can be compared over time. Simple alert keys such as alerts:slow_response are set when thresholds are exceeded.

Screenshot 11: Error tracking and categorization

```
> HGETALL metrics:errors:counts
1) "db_timeout"
2) "3"
3) "auth_failure"
4) "5"
5) "validation_error"
6) "2"
7) "rate_limit"
8) "4"
> ZREVRANGE metrics:errors:by_type 0 -1 WITHSCORES
1) "auth_failure"
2) "5"
3) "rate_limit"
4) "4"
5) "db_timeout"
6) "3"
7) "validation_error"
8) "2"
```

Error counts are tracked in the hash `metrics:errors:counts`, where fields are incremented with `HINCRBY`. `HGETALL` shows the total counts per error type. A Sorted Set, `metrics:errors:by_type` was created so `ZREVRANGE` can rank error categories by frequency and highlight the most common problems.

Part 6: Analytics Dashboard

Screenshot 12, 13, 14: Complete analytics dashboard with all key metrics

```
> ZREVRANGE metrics:errors:by_type 0 -1 WITHSCORES
1) "auth_failure"
2) "5"
3) "rate_limit"
4) "4"
5) "db_timeout"
6) "3"
7) "validation_error"
8) "2"
```

```
> HGETALL analytics:features:global
1) "leaderboard_views"
2) "15"
3) "profile_updates"
4) "5"
5) "settings_changes"
6) "3"
7) "friends_actions"
8) "6"
9) "messages_sent"
> SCARD analytics:new_players:2024-03-11
(integer) 2
> SCARD analytics:retention:day1:2024-03-11
(integer) 1
> SCARD analytics:retention:day3:2024-03-11
(integer) 0
> SCARD analytics:retention:day7:2024-03-11
(integer) 1
```

The final dashboard ties together player behavior, revenue, feature usage, and system health using a small set of queries. WAU is measured with SCARD, while player retention for analytics:new_players:2024-03-11 is derived from the analytics:retention:* sets. Monetization and loyalty are summarized with ZREVRANGE revenue:players(top spenders) and ZREVRANGE analytics:streaks (longest login streaks). HGETALL analytics:features:global and ZREVRANGE analytics:game_modes show feature usage and game mode popularity, and HGETALL metrics:system:current plus the error metrics provide a quick operational health view. KEYS alerts:* surfaces any active alerts on performance or errors.

Summary Report

All four players appear as weekly active users and Alice is the most engaged (4/5 days), with Carol and David close behind (3/5 days). Player analysis for new players shows solid Day 1 and Day 7 retention but a dip on Day 3, indicating a mid-week drop-off. Alice and Carol are also the top spenders. Leaderboard views, ranked mode, and messaging are the most popular features. System metrics show healthy player counts and acceptable response times, but authentication and rate-limit errors occur more often than other error types.

Recommendations Based on Data

Focus on improving Day-3 engagement with re-engagement rewards or quests, since this is where retention drops. Invest in competitive and social features (ranked mode, leaderboards, messaging), which already show the strongest usage. Use streak and revenue data to reward loyal players with targeted bonuses or VIP perks. Finally, refine authentication and rate-limit handling to reduce those error categories and continue monitoring alerts as the player base grows.